ComicCraft - AI Comic Story Creator using Gemini Models

TEAM MEMBERS:

Vasanthakumaran.A- vasanthakumaran125@gmail.com
Janani.G -jananig06112007@gmail.com
kanimozhi.S - sasvika1503@rediffmail.com
lakshmi.S -lakshmisellamuthu1234@gmail.com

Project Description:

ComicCraft is a web-based application that uses AI to generate personalized comic book stories and illustrations based on user-provided prompts. Built with FastAPI and integrated with Google's Gemini AI models along with Stable Diffusion, ComicCraft streamlines the creative process of generating storylines, dialogues, and vivid comic-style imagery automatically.

The application takes in user information—such as story prompt, main character name, setting, tone, and art style—and generates a panel-by-panel storyline and corresponding illustrations. Users can view the comic preview directly on the web interface and also download the full comic in PDF format, structured with both images and narration.

With a clean, interactive HTML interface and efficient image generation powered by Hugging Face's Diffusers library, ComicCraft ensures that each story feels cohesive and visually engaging. It offers both API endpoints and an intuitive frontend for ease of use, making comic creation accessible even to non-artists and casual users.

Scenarios:

Scenario 1 : A user enters a prompt describing their comic story, such as "A brave fox exploring an enchanted forest." The system uses this input to generate a complete comic strip, including multiple panels with accompanying images and text. The user also specifies preferences like the main character's name, the setting (e.g., forest), tone (e.g., dramatic), and art style (e.g., anime). The system combines Gemini 1.5 Pro for story generation and Stable Diffusion for image creation to produce a visually immersive, personalized comic strip. This personalized approach enhances user satisfaction by generating a cohesive comic story that fits the user's creative vision.

Scenario 2: A user wants a light-hearted, cartoonish feel for their comic. On the input form, they select "funny" as the tone and "comic book" as the art style (instead of the default or their previous choices). When they submit, the system regenerates the entire pipeline—outline, story, and images—this time with prompts and model instructions tailored to produce a more humorous narrative and a classic comic-book look. This lets users iterate on and customize the mood and visual style of their comic until it matches their creative vision.

Scenario 3: After reviewing the on-screen preview of their comic, the user decides to download it.
The system then:

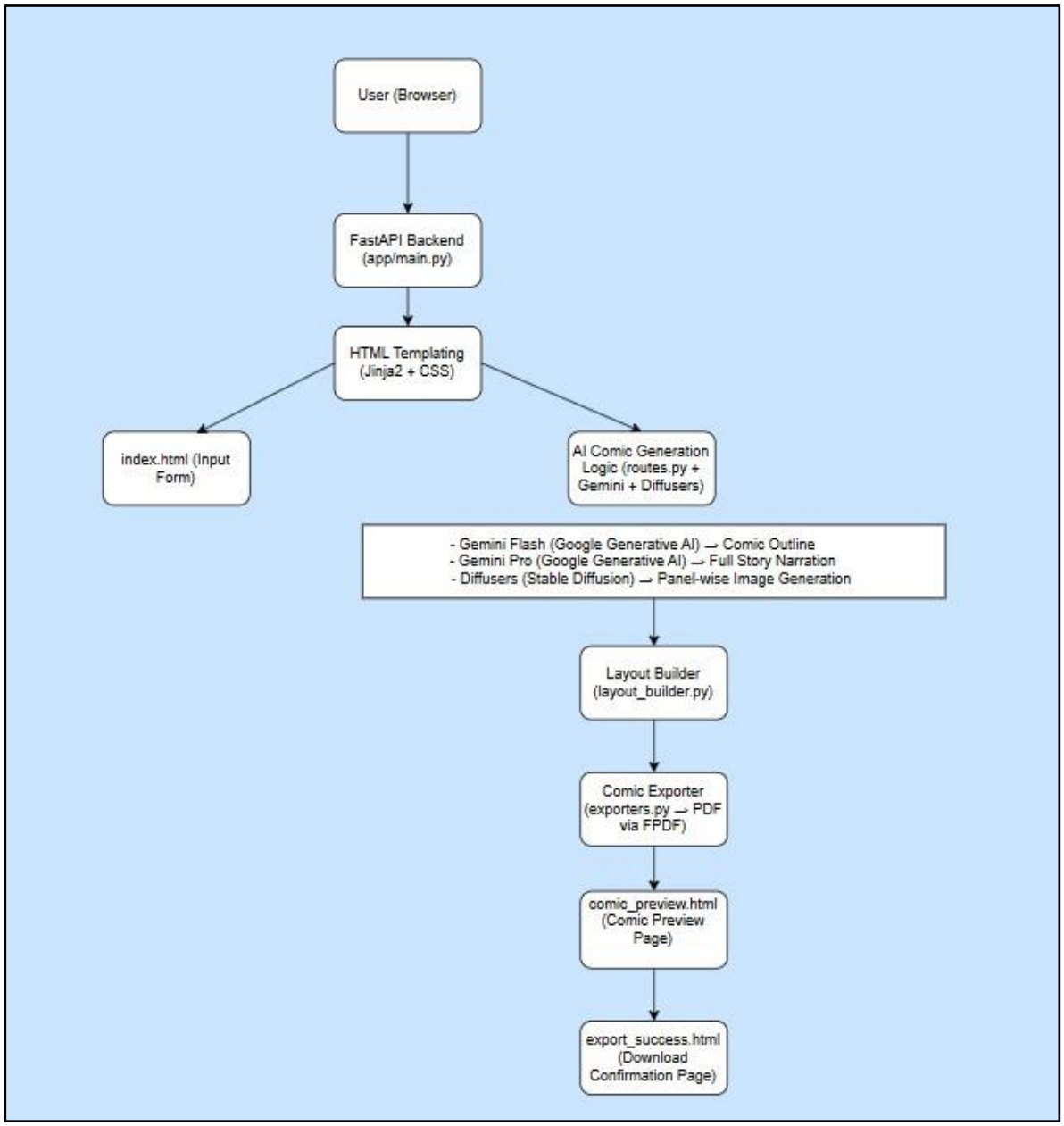
1. Layout Binding: Uses layout_builder.py to assemble each panel's title, image, and text into a structured layout.

2. **PDF Generation**: Calls exporters.py to compile the comic into a **PDF** file using **FPDF**, placing images and narrative text onto consecutive pages and saving the PDF with a timestamped

filename.

3. Export Success: The user is redirected to an export success page, confirming the successful download. This feature ensures users can save, print, or share their comic in a professional PDF format.

Technical Architecture:



Pre - requisites:

1. FastAPI Framework Knowledge: FastAPI Documentation
2. Uvicorn ASGI Server: Uvicorn Docs
3. Python Programming Proficiency: Python Official Docs
4. HTML, CSS, and Jinja2 Template Skills: W3Schools HTML/CSS/Jinja2 Tutorials
5. Google Gemini API Familiarity: Google Generative AI Documentation
6. Hugging Face Diffusers & Stable Diffusion: Hugging Face Diffusers Documentation
7. PyTorch Basics: PyTorch Documentation
8. PDF Generation with FPDF: FPDF Documentation
9. Version Control with Git:Git Documentation
10. Python Package & Environment Setup (pip, virtualenv, or conda): Virtualenv Guide

Project Workflow:

Milestone 1. Model selection and Architecture

- Activity 1.1. Research and Select the Appropriate Generative AI Model
- Activity 1.2. Define the architecture of the application
- Activity 1.3. Set up the development Environment

Milestone 2. Core functionalities Development

- Activity 2.1. Develop the core functionalities
- Activity 2.2. Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 3. route.py Development

- Activity 3.1. Writing the Main Application Logic in routes.py

Milestone 4. Frontend Development

- Activity 4.1. Designing and Developing User Interface
- Activity 4.2. Creating Dynamic Templates with FastAPI's Jinja2

Milestone 5. Deployment

- Activity 5.1. Preparing the Application for Local Deployment
- Activity 5.2. Testing and Verifying Local Deployment

Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting appropriate generative AI models for the ComicCraft application. This involves researching the capabilities and performance of multiple AI models to ensure that the chosen models align well with the application's objectives of generating comic storylines, character dialogues, scene descriptions, and comic-style illustrations.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

Review the specific needs of the ComicCraft application, focusing on the types of content it must generate:

- Structured comic outlines (panel-by-panel)
- Full comic narration and character dialogues
- Visual comic-style illustrations for each panel

The system must deliver creative, engaging, and visually rich comic outputs for users based on their custom story prompts.

Model Evaluation

Research documentation and capabilities of:

- **Google's Gemini Models**: Gemini Flash (optimized for fast structured outputs) and Gemini Pro (optimized for detailed, creative text generation).
- **Hugging Face Diffusers**: Stable Diffusion model (for high-quality AI-generated images based on prompts).

Compare the selected models based on:

- Response quality for creative writing tasks (narration and dialogues)
- Prompt adherence and richness of detail for scene descriptions
- Speed and resolution of image generation
- Model stability, available documentation, and community support

Practical testing was conducted using different prompts to assess each model's storytelling abilities and image generation performance.

Final Model Selection

Choose the AI models that best match ComicCraft's project needs:

- **Gemini Flash (models/gemini-1.5-flash)**:
  Selected for fast and structured generation of comic panel outlines based on story prompts.
- **Gemini Pro (models/gemini-1.5-pro)**:
  Selected for detailed, creative comic narration and engaging character dialogues.
- **Stable Diffusion (runwayml/stable-diffusion-v1-5)**:
  Selected for generating high-quality comic-style illustrations from image prompts.

This combination ensures that ComicCraft can deliver rich, creative storytelling alongside vivid, visually appealing comic panels in an automated, efficient manner.

Activity 1.2.Define the architecture of the application

Architecture Overview: The architecture of ComicCraft is structured into three primary components:

- **Frontend** (HTML, CSS, Jinja2)
- **Backend** (FastAPI application)
- **AI Integration** (Google Gemini APIs and Hugging Face Diffusers)

The frontend captures user inputs and displays generated comics, the backend handles data processing and routing, and the AI services generate story content and illustrations.
Below is a high-level description of each component's role in the system.

Frontend Responsibilities:

- Users interact with a simple and user-friendly web interface developed using **HTML, CSS, and Jinja2 templates**.
- Key user input fields include:
  - **Story Prompt**: Main idea for the comic
  - **Character Name**: Hero of the comic
  - **Setting**: Location (e.g., forest, school, city)
  - **Story Tone**: Mood of the story (e.g., dramatic, funny)
  - **Art Style**: Visual style preference (e.g., anime, realistic)
- Upon form submission, data is sent to the FastAPI backend via a POST request for processing.

Templates Used:

- index.html: For user input collection
- comic_preview.html: To display the generated comic
- export_success.html: To show a success message after comic export

The frontend ensures a seamless flow from input collection to comic generation display.

Backend Responsibilities:

- FastAPI is used to manage server-side operations and route handling.
- Key backend tasks:
  - Receive form data from the frontend or JSON API calls.
  - Call AI models to generate panel outlines, story narration, and illustrations.
  - Organize and build the complete comic layout.
  - Export the comic to a downloadable PDF format.
- Routes are defined in routes.py to handle:
  - Form-based and JSON-based comic generation
  - Image testing
  - Comic export confirmation

The backend ensures proper coordination between user inputs, AI generation workflows, and template rendering.

AI Integration Points:

- Gemini Flash is called to generate the structured 5-panel comic outline based on the user's story prompt.
- Gemini Pro is called next to create detailed narration and character dialogue from the outline.
- Stable Diffusion is used to generate comic-style illustrations based on the AI-generated image prompts for each panel.
- The backend receives AI responses, formats the text and images appropriately, and passes them to the frontend templates for preview and export.

All AI model interactions happen dynamically during comic generation, ensuring that every comic is fully unique and customized based on user input.

Activity 1.3 Set up the development Environment:

Install Python & Pip:Ensure that Python is installed on your system along with pip, which will be used to manage the necessary project dependencies.

Create Virtual Environment:This ensures that project-specific packages do not conflict with system-wide installations.

```
python -m venv comiccraft-env

comiccraft-env\Scripts\activate  # For Windows
```

Install Required Libraries:

```
pip install fastapi uvicorn jinja2  python-multipart google-generativeai diffusers transformers fpdf Pillow accelerate
```

Run Server:

uvicorn app.main:app --reload
Visit http://1 27.0.0.1 :8000 for the app and /docs for API testing.

Set Up Project Structure:

Create the initial project directory structure for ComicCraft:

```
comiccraft/
│
├── app/
│   ├── main.py
│   ├── routes.py
│   ├── gemini_flash.py
│   ├── gemini_pro.py
│   ├── image_generator.py
│   ├── layout_builder.py
│   └── exporters.py
│
├── templates/
│   ├── index.html
│   ├── comic_preview.html
│   └── export_success.html
│
├── static/
│   ├── panels/        # Generated panel images
│   ├── exports/       # Exported comic PDFs
│   └── fonts/         # Fonts for PDF generation
│
├── .env               # Environment variables (API keys)
└── requirements.txt   # Python dependencies list
```

Milestone 2. Core functionalities Development

Activity 2.1 Develop the core functionalities:

Generate Structured Comic Panel Outline

Function: generate_outline()
File: gemini_flash.py

- This function uses the GeminiFlash AI model to generate a structured 5-panel comic outline based on the user's story prompt.
- Each panel contains a panel number, title, scene description, and an image generation prompt.
- The output is a list of dictionaries, one for each comic panel.

```
def generate_outline(user_prompt: str) -> list:
    """
    Generates a 5-panel comic layout based on the user's story idea using Gemini.

    Args:
        user_prompt (str): The user's comic idea prompt.

    Returns:
        list: A list of dictionaries, one for each panel.
    """
    prompt = f"""
You are a professional AI comic planner.

Your task is to generate a *strictly formatted* JSON array containing 5 panel descriptions for a comic based on the story idea below:

STORY: "{user_prompt}"

Each JSON object must include:
- "panel" (integer)
- "title" (string)
- "scene_description" (string)
- "image_prompt" (string)

Respond ONLY in this valid JSON format, without any explanations or markdown:
[
  {{
    "panel": 1,
    "title": "Title here",
    "scene_description": "Scene description here",
    "image_prompt": "Image prompt for Stable Diffusion"
  }},
  ...
]
"""
```

```
try:
    response = model.generate_content(prompt)
    output_text = response.text.strip()

    print("\n🔥 RAW GEMINI RESPONSE 🔥\n", output_text)

    # Remove any markdown formatting if present
    if output_text.startswith("```json"):
        output_text = output_text.replace("```json", "").replace("```", "").strip()

    panel_data = json.loads(output_text)

    # Additional structure validation
    if not isinstance(panel_data, list):
        raise ValueError("Gemini response is not a list.")

    for panel in panel_data:
        if not isinstance(panel, dict) or not all(key in panel for key in ("panel", "title", "scene_description", "image_prompt")):
            raise ValueError(f"Invalid panel format or missing keys: {panel}")

    return panel_data

except json.JSONDecodeError as e:
    print("❌ JSON Decode Error:", e)
    print("❌ Full Text Received:\n", output_text)
    return [{"error": f"JSON parsing failed: {str(e)}"}]

except Exception as e:
    print("❌ Unexpected Error:", e)
    return [{"error": f"Generation failed: {str(e)}"}]
```

Generate Comic Story Narration and Dialogue

Function: generate_story()
File: gemini_pro.py

- This function uses the **Gemini Pro** AI model to expand the panel outlines into a full comic-style story.
- It creates engaging narration and character dialogues for each panel.
- The output is a single formatted text containing all the panels' stories.

```python
def generate_story(outline: list) -> str:
    """
    Generates a detailed comic story with narration and character dialogue
    from a list of comic panel outlines using Gemini 1.5 Pro.

    Args:
        outline (list): A list of strings representing each comic panel's idea.

    Returns:
        str: The generated comic story text or an error message.
    """

    # Format the panel outline as a numbered list for clarity
    formatted_outline = "\n".join([f"{i+1}. {item}" for i, item in enumerate(outline)])

    # Construct the prompt
    prompt = f"""
You're a comic book writer.

Given the following panel breakdown, write a comic-style story with engaging narration and character dialogues for each panel.

Panel Outline:
{formatted_outline}

Guidelines:
- Use a fun and engaging tone, like an actual comic book.
- Include narration and clearly marked character lines.
- Keep each panel self-contained but part of a cohesive story.
"""

    try:
        response = model.generate_content(prompt)
        return response.text
    except Exception as e:
        return f"Error generating story: {str(e)}"
```

Generate Comic Illustrations from Prompts

Function: generate_image()
File: image_generator.py

- This function uses the **Stable Diffusion** model to create a comic-style image based on the provided image prompt.
- It sanitizes the prompt for safe filenames and saves the generated image to the static/panels directory.
- Returns the file path to the saved image.

```python
def generate_image(prompt, filename=None):
    if not filename:
        filename = sanitize_filename(prompt)

    image = pipe(prompt).images[0]
    path = f"static/panels/{filename}"
    os.makedirs(os.path.dirname(path), exist_ok=True)
    image.save(path)
    return path
```

Organize Comic Panels into Layout Structure

Functions: build_comic_layout()

File: layout_builder.py

- This function organizes the generated images and full comic story into a structured layout.
- It matches each image with its corresponding panel story.
- The output is a list of dictionaries containing the panel number, image path, and text.

```python
def build_comic_layout(image_paths, full_story, outline):
    # Split the full story into individual panel segments
    story_panels = full_story.split("**Panel")
    story_panels = [f"**Panel{panel}" for panel in story_panels if panel.strip()]

    # Build layout panel by panel
    layout = []
    for idx, (image, text, panel_info) in enumerate(zip(image_paths, story_panels, outline), start=1):
        layout.append({
            "panel": idx,
            "title": panel_info.get("title", f"Panel {idx}"),
            "image_path": image,
            "text": "\n".join(text.strip().splitlines()[1:]).strip(),  # remove panel title
            "scene_description": panel_info.get("scene_description", "")
        })

    return layout
```

Export Comic Panel and Story into PDF

Function: save_pdf()
File: exporters.py

- This function compiles the full comic into a multi-page PDF file using the FPDF library.
- Each panel's image and narration are placed neatly on separate pages.
- The final PDF file is saved into the static/exports folder, and the path is returned.

```python
def save_pdf(layout):
    pdf = PDF()
    pdf.set_auto_page_break(auto=True, margin=15)

    # Add Unicode font
    pdf.add_font('DejaVu', '', FONT_PATH, uni=True)
    pdf.set_font('DejaVu', '', 12)

    for panel in layout:
        image_path = panel['image_path']
        story_text = panel['text']

        pdf.add_page()

        # Panel title (no bold)
        pdf.set_font('DejaVu', '', 14)
        pdf.cell(0, 10, f"Panel {panel['panel']}", ln=True, align="C")
        pdf.set_font('DejaVu', '', 12)

        # Image placement
        y_image = 30
        image_height = 100
        spacing_after_image = 15

        if os.path.exists(image_path):
            pdf.image(image_path, x=10, y=y_image, w=pdf.w - 20, h=image_height)
        else:
            pdf.set_y(y_image)
            pdf.multi_cell(0, 10, f"Image missing: {image_path}")

        # Text placement below image
        pdf.set_y(y_image + image_height + spacing_after_image)
        story_lines = story_text.strip().splitlines()
        # Remove title line like "**Panel 1: Title**" if present
        if story_lines and story_lines[0].strip().lower().startswith("**panel"):
            story_lines = story_lines[1:]
        cleaned_text = "\n".join(story_lines).strip()
        pdf.multi_cell(0, 10, cleaned_text)

    # Save with timestamp
    timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    filename = f"comic_{timestamp}.pdf"
    pdf_path = os.path.join(EXPORT_FOLDER, filename)
    pdf.output(pdf_path)

    return pdf_path
```

Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

Define Routes in FastAPI

All routing logic is handled in routes.py.
Each route is linked to the corresponding backend function that processes the comic generation workflow developed in Activity 2.1.

Process User Input

- HTML Form: Created in index.html
- Users enter: story prompt, main character name, setting, tone, and art style
- FastAPI captures inputs using Form(...) parameters from the request
- Pydantic schema (PromptRequest) is used to validate JSON input for API-based comic generation

Integrate Gemini API Calls

Each route handler connects directly to the respective AI-powered function modules:

- /generate →
  - Calls generate_outline() to create panel outlines from user prompt
  - Calls generate_story() to expand into narration and dialogue
  - Calls generate_image() for each panel to create comic-style images
  - Calls build_comic_layout() to organize the final layout
  - Calls save_pdf() to compile and export the complete comic into a PDF
- /generate-comic/json →
  - Similar flow as /generate, but triggered via a raw JSON API request
  - Returns JSON response with layout data and PDF path for API clients
- /test-image →
  - Calls generate_image() directly with a custom prompt to test image generation functionality separately

All AI-generated responses (text and images) are processed, cleaned, and injected into the Jinja2 frontend templates (comic_preview.html, export_success.html) for a seamless user experience.

Milestone 3. routes.py Development

In this milestone, the primary focus was on setting up all major FastAPI routes for the app, handling user inputs efficiently, integrating AI generation workflows, and ensuring smooth interaction between backend processing and frontend templates.
This step-by-step approach created a stable foundation for processing prompts, generating comics, and providing a seamless user experience.

Activity 3.1: Writing the Main ApplicationLogic in routes.py

Defining Core routes in routes.py:

Separate routes were established for each of ComicCraft's core functionalities:

- / → Loads the **homepage** (index.html) where users can submit their story details.
- /generate → Handles **form submission**, processes the input using AI, generates the comic panels, and returns the **comic preview** page.
- /generate-comic/json → An API route that accepts **JSON payloads**, triggers comic generation, and returns **comic layout data** and the generated PDF path.
- /export-success → Displays a **success confirmation page** after the comic is downloaded.
- /test-image → A developer utility route to **test image generation** from a direct prompt without full comic creation.

Each route serves as a distinct entry point for different types of user interaction — through browser forms or direct API requests.

Set up Route Handlers for Each Feature

Each route handler captures and processes user inputs using FastAPI's form and request body mechanisms:

- For /generate, user inputs are captured using Form(...) parameters for:
  - Story prompt
  - Character name
  - Setting
  - Story tone
  - Art style
- For /generate-comic/json, inputs are captured from a JSON body using a PromptRequest model with Pydantic.

Form data and JSON payloads are correctly extracted and passed into backend functions (generate_outline, generate_story, generate_image, build_comic_layout, save_pdf) ensuring smooth AI processing based on user details.

The route handlers manage error handling via FastAPI's HTTPException system and use try-except blocks to gracefully handle unexpected failures like API errors or JSON decode errors.

Integrate AI workflows in each Route

Within each relevant route, the core AI functionalities were integrated: In

/generate and /generate-comic/json:

- generate_outline() is called to create a 5-panel structured story.
- generate_story() is called to write narration and dialogues.
- generate_image() is called iteratively to generate images for each panel.
- build_comic_layout() structures the final panel layout.
- save_pdf() compiles the story and illustrations into an exportable PDF.

Each AI-generated output is processed and dynamically passed to HTML templates (comic_preview.html) or returned as a JSON response for API clients.

This ensures that users receive a complete, coherent comic experience, while developers and testers can interact with the backend directly through structured API responses.

Comic Generation Route:

```python

@router.post("/generate", response_class=HTMLResponse)
async def generate_comic(
    request: Request,
    prompt: str = Form(...),
    character_name: str = Form(...),
    setting: str = Form(...),
    tone: str = Form(...),
    style: str = Form(...)
):
    try:
        # Combine user input into a single full prompt
        full_prompt = (
            f"{prompt}\n"
            f"The main character is {character_name}. "
            f"The setting is a {setting}. "
            f"The tone is {tone}. The art style is {style}."
        )

        # Step 1: Generate panel outline
        outline = generate_outline(full_prompt)

        if not isinstance(outline, list) or not all("image_prompt" in panel for panel in outline):
            raise ValueError("Invalid outline structure from Gemini response.")

        # Step 2: Generate story
        full_story = generate_story(outline)

        # Step 3: Generate images
        images = [generate_image(panel["image_prompt"]) for panel in outline]

        # Step 4: Build layout
        layout = build_comic_layout(images, full_story, outline)

        # Step 5: Export to PDF
        pdf_path = save_pdf(layout)
        web_pdf_path = "/" + pdf_path.replace("\\", "/")  # Make path safe for web

        return templates.TemplateResponse("comic_preview.html", {
            "request": request,
            "layout": layout,
            "pdf_path": web_pdf_path
        })

    except Exception as e:
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))

```

Export Success Route:

```python

@router.get("/export-success", response_class=HTMLResponse)
async def export_success(request: Request, pdf_path: str):
    return templates.TemplateResponse("export_success.html", {
        "request": request,
        "pdf_path": pdf_path
    })


```

Image generation Route:

```python

@router.get("/test-image")
async def test_image(prompt: str = "A futuristic city at sunset, sci-fi, cinematic, artstation"):
    try:
        image_path = generate_image(prompt)
        return {"message": "Image generated successfully", "path": image_path}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

Milestone 4. Frontend Development

This part involved building responsive and clean web pages using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring that users can easily submit inputs, view generated comics panel-by-panel, and confirm comic export actions.
This milestone guarantees an engaging and seamless comic creation experience for all users.

Activity 4.1 Designing and Developing the User Interface

Set Up the Base HTML Structure

Developed index.html as the main entry point for user input.
Structured the form to capture:

- Story Prompt
- Main Character Name
- Setting
- Story Tone
- Art Style

Each field is properly labeled and grouped using semantic HTML elements.
The design uses a scenic background image, simple typography, and organized form layout to encourage creativity and focus on storytelling.
Navigation is minimal to maintain focus, with a direct flow from input to comic generation.

Design a Responsive Layout Using CSS

Embedded CSS styles were applied directly within each HTML file.
Layouts use fixed width with centered content to ensure readability across different screen sizes.
Consistent button styling with hover effects was implemented.
Input fields and result panels are designed with padding, box shadows, and rounded corners for a clean, modern comic-generation experience.
The light color scheme enhances visibility and promotes an inviting visual aesthetic.

Create Separate Pages for Each Core Functionality

Three Jinja2-powered HTML templates were created under the /templates folder:

- index.html:
  A user-friendly form where users input the story prompt, character name, setting, tone, and preferred art style.
  Submits data to the /generate FastAPI route via POST.
- comic_preview.html:
  Displays the AI-generated comic panel-by-panel.
  Each panel includes:
  - Title
  - Scene illustration (image)
  - Scene description (in italics)
  - Captions and narration text This allows users to review the full comic storyline sequentially.
- export_success.html:
  Displays a success message after the user downloads their comic as a PDF.
  Encourages users to return and create a new comic.

Templates are modular, cleanly separated by functionality, and UI transitions smoothly between comic creation and comic review.

Activity 4.2 Creating Dynamic Templates with FastAPI's Jinja2

Integrate Jinja2 Templating for Dynamic Content Rendering

- Used FastAPI's Jinja2Templates for template rendering:

```
from fastapi.templating import Jinja2Templates templates =
Jinja2Templates(directory="templates")
```

Each FastAPI route uses TemplateResponse to dynamically render HTML templates based on the user's interaction:

- / route renders index.html
- /generate route renders comic_preview.html
- /export-success route renders export_success.html

This dynamic binding ensures that the comic content (layout, images, text) updates automatically after user input.

Bind Backend Data to HTML Templates

> index.html

- Form elements use name="..." attributes that directly map to backend Form(...) parameters in FastAPI.
- Submits form data via POST to /generate.

> comic_preview.html

- Dynamically displays:
  - Panel number, title
  - Image generated for the panel
  - Scene description (in italics)
  - Captions and narration text
- Loops through the layout list using Jinja2 {% for panel in layout %} syntax.

> export_success.html

- Displays a confirmation message using Jinja2 variables.
- Receives the exported PDF path as context and optionally provides a button for returning to the homepage.

The frontend of ComicCraft is fully integrated with FastAPI, dynamically renders AI-generated comics, and provides a smooth user flow from prompt input to comic export.

Milestone 5. Deployment

Now, the focus is on deploying the ComicCraft application on a local system using FastAPI. This involves configuring the server environment, installing all required dependencies, setting up static files and templates, and launching the app locally to simulate a real-world environment.
This milestone verifies that the complete comic generation workflow — including AI text generation, image creation, layout building, and PDF export — runs smoothly before planning deployment to a cloud platform in the future.

Activity 5.1: Preparing the Application for Local Deployment

Set up a virtual environment

Begin by creating a virtual environment to manage dependencies and ensure the deployment is isolated from other Python projects. Activate the environment and install all required libraries listed in the requirements.txt file, including FastAPI, Uvicorn, Google Generative AI, Hugging Face Diffusers, FPDF, and others.

```
python -m venv env #
For Windows
env\Scripts\activate #
For macOS/Linux
source env/bin/activate
```

```
pip install -r requirements.txt
```

Configure environment variables

- Use environment variables to store sensitive data such as the Gemini API Key and Hugging Face API token
- These variables can be set directly in the terminal or added to a .env file which is loaded into the application using python-dotenv.

```
GEMINI_API_KEY=your_gemini_api_key_here
HF_API_KEY=your-huggingface-api-key-here
```

Frontend-Backend Rendering via Jinja2

All user interactions are facilitated via Jinja2-rendered templates:

- index.html: Collects user inputs
- result.html: Displays the generated AI plan
- all_users.html: Lists all user records for admin view

Activity 5.2: Testing and Verifying Local Deployment

Start the FASTAPI Server

Navigate to the project root where your app/main.py file is located.
Use Uvicorn to launch the FastAPI development server with hot reload enabled:

```
uvicorn app.main:app --reload
```

Access the Application Locally

Once the server is running, open any web browser and navigate to: http://1

27.0.0.1 :8000

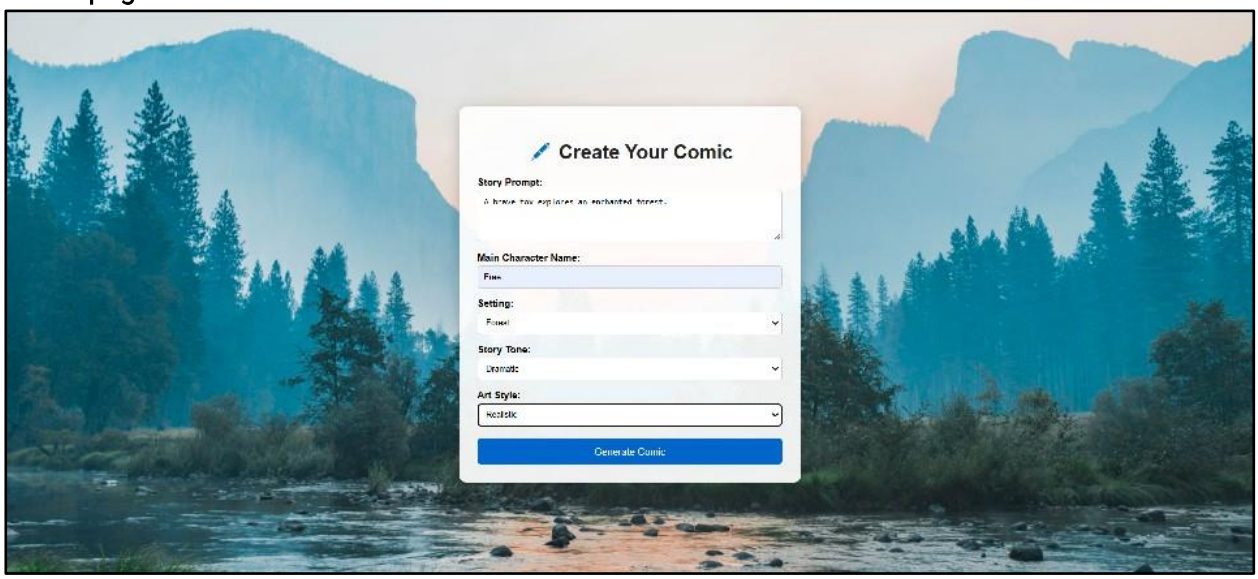
This URL loads the ComicCraft homepage, where users can input their story prompt, character name, setting, tone, and art style to generate a personalized AI-powered comic.

Additionally, the interactive API documentation provided by FastAPI can be accessed at:

http://1 27.0.0.1 :8000/docs
This documentation allows developers to test available API endpoints, such as /generate-comic/json, /generate, /export-success, and /test-image.

Exploring Website's web pages:

Home page:



Description:

The homepage of ComicCraft is designed as an intuitive starting point for users to create AI-generated comic stories. It features a clean layout with a scenic background that fosters a creative atmosphere.

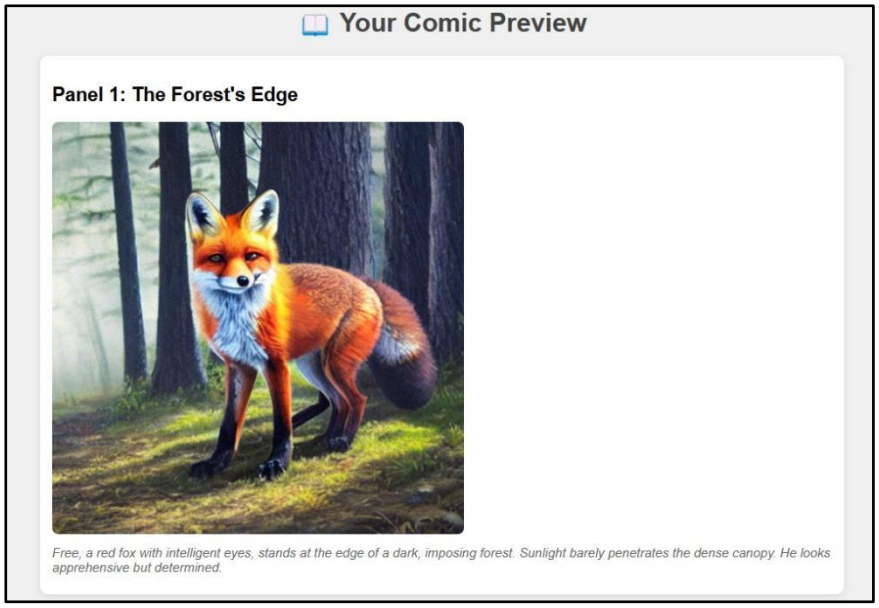
At the center, a structured form collects user inputs necessary for comic generation. Users can:

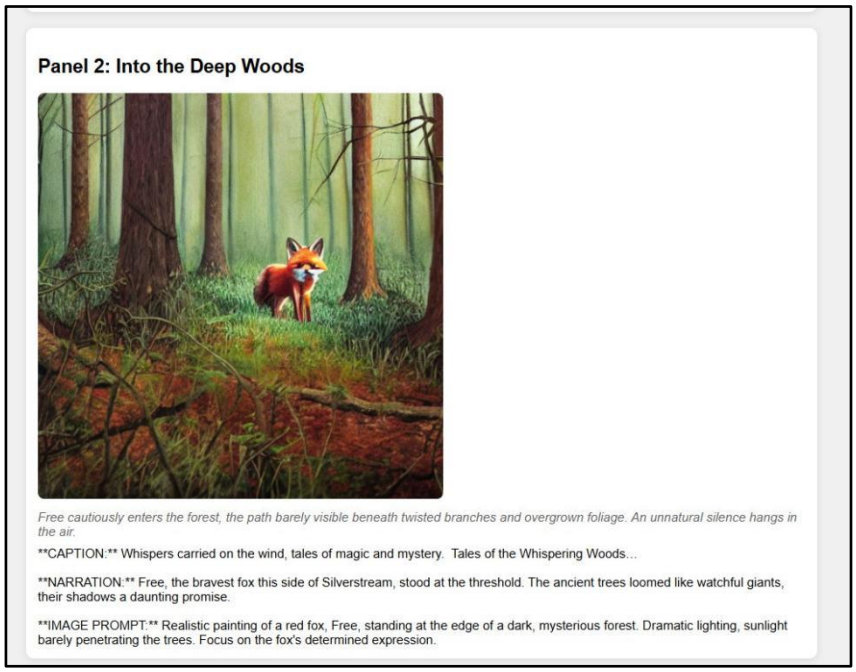
- Enter a Story Prompt describing their comic idea,
- Specify a Main Character Name,
- Choose a Setting (school, forest, space, city),
- Select a Story Tone (light-hearted, dramatic, poetic, funny),
- Pick an Art Style (anime, pixel art, comic book, realistic).

Once the form is submitted, the system uses AI models to automatically generate a complete comic, including storyline outlines, detailed dialogues, illustrations, and an exportable PDF.

The homepage is designed to be simple, user-friendly, and visually appealing, encouraging creativity and making the comic creation process accessible to all users without requiring technical expertise.

Comic Preview- Result page:





Description:

The Comic Preview Page in ComicCraft presents the AI-generated comic panels sequentially after the user submits their input through the homepage form. This page provides a complete view of the comic's storyline, combining visual and textual elements for each panel.

Each panel displayed on the page includes:

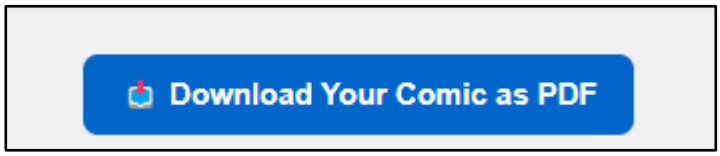
- Panel Title: Clearly labeled with the panel number and a descriptive title (e.g., "Panel 2: Into the Deep Woods").
- Comic Image: An AI-generated illustration created based on the scene's image prompt.
- Scene Description: A short descriptive paragraph, displayed in italics, that sets the atmosphere and context for the panel.
- Caption and Narration:
    - Caption providing a brief, ambient description of the environment or background sounds.
    - Narration detailing the main character's actions, emotions, or dialogue within the scene.
- Image Prompt Reference: The original artistic description used to generate the illustration, displayed for reference.

The design of the Comic Preview Page ensures that each panel is clearly separated and visually

appealing, making it easy for users to follow the story's flow panel-by-panel.
The combination of imagery, scene-setting description, and narrative text helps create an immersive storytelling experience directly from the AI-generated content.

At this stage, the page focuses purely on comic viewing and story review, allowing users to experience the full creative output before proceeding to any final actions like exporting.

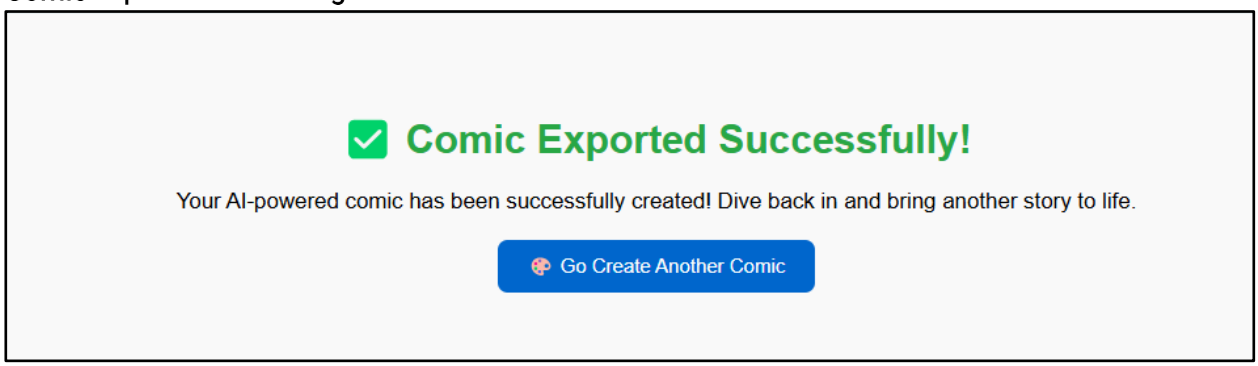
Download Button:



Description:

On the Comic Preview Page, a "Download Your Comic as PDF" button is displayed.
When users click this button, the entire comic — including images, panel titles, scene descriptions, captions, and narration — is compiled into a professionally formatted PDF file and downloaded directly to the user's device.
This ensures users can save, share, and view their comic offline.

Comic Export Success Page:



Description:

After the download is triggered, users are automatically redirected to the Comic Export Success Page. This page provides a confirmation message indicating that their AI-powered comic has been successfully created and downloaded. The success page also includes a call-to-action button, "Go Create Another Comic", which encourages users to return to the homepage and start a new comic creation process.

The export and success flow are designed to provide closure and satisfaction after the creative process, ensuring users have a tangible output from their story idea while promoting continued engagement with the platform.

Conclusion:

ComicCraft successfully demonstrates the creative potential of AI-driven comic generation. By combining the capabilities of Large Language Models (Gemini Flash and Gemini Pro) for storytelling and Stable Diffusion for comic-style illustration, the application enables users to transform simple prompts into complete, visually engaging comic narratives.

The project provides an end-to-end experience — from personalized story outline generation, full narration, high-quality image creation, to exporting the final comic as a downloadable PDF — all within an intuitive web interface. ComicCraft simplifies comic creation for non-artists, storytellers, and hobbyists, offering a seamless blend of imagination and AI assistance.

While the current version focuses on single comic generation without user account management, the strong modular architecture allows easy future enhancements like user profiles, comic libraries, or multi-page story arcs.

Overall, ComicCraft showcases how AI can democratize storytelling and empower users to bring their creative visions to life with minimal effort.